

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas
Escuela Profesional de Ingeniería de Software



SW505 – CONSTRUCCIÓN DE SOFTWARE I PLAN DE IMPLEMENTACIÓN Y DESPLIEGUE

P2 – Plataforma de Reservas de Servicios
(Similar a Calendly / SimplyBook / Booksy)

Semana 9 – Entregable Oficial
Docente: Mag. Juan Luis Herencia Guerra
Lima, 2026

PARTE 1: PLAN DE IMPLEMENTACIÓN

1.1 Estrategia General de Implementación

La implementación de la Plataforma de Reservas de Servicios (P2) adopta un enfoque iterativo e incremental, fundamentado en los principios del Proceso Unificado de Rational (RUP) y las prácticas modernas de ingeniería de software ágil. Esta estrategia contrasta deliberadamente con un enfoque waterfall monolítico, dado que la naturaleza modular de la arquitectura del sistema —organizada en capas desacopladas con separación CQS estricta— permite que cada módulo funcional sea construido, integrado y validado de forma independiente antes de proceder con la siguiente iteración.

Principios Rectores de la Estrategia

- Construcción vertical (slice by feature): Cada iteración entrega una funcionalidad completa y verificable de punta a punta, desde la base de datos hasta la interfaz de usuario, evitando la integración tardía de componentes.
- Priorización por riesgo técnico: Los módulos con mayor complejidad técnica (conurrencia en reservas, autenticación JWT) son implementados en etapas tempranas, permitiendo identificar y resolver problemas estructurales antes de que el sistema alcance mayor complejidad.
- Independencia de despliegue: Cada módulo puede ser desplegado, actualizado y revertido de forma independiente mediante contenedores Docker, reduciendo el radio de blast ante fallos en producción.
- Trazabilidad continua: Toda decisión de implementación debe ser trazable hasta un Requerimiento Funcional (RF) o No Funcional (RNF) específico del documento de análisis y diseño.
- Integración continua desde el día uno: Los pipelines CI/CD son configurados al inicio del proyecto, no al final, garantizando retroalimentación temprana sobre la calidad del código.

Justificación del Orden de Implementación

El orden de construcción de los módulos responde a un análisis de dependencias técnicas y reducción de riesgos. El módulo de Autenticación y Gestión de Perfiles es implementado primero porque constituye la fundación de seguridad sobre la que todos los demás módulos dependen: sin mecanismos JWT funcionales, no es posible proteger ningún endpoint del sistema. A continuación, el módulo de Disponibilidad y Agenda provee la estructura de datos central sobre la que opera el módulo de Agendamiento. Este último, siendo el núcleo del negocio y el de mayor complejidad técnica (conurrencia, bloqueos, transacciones ACID), es construido sobre una base estable. Finalmente, el módulo de Notificaciones es implementado como un componente desacoplado que consume eventos de dominio, permitiendo su desarrollo paralelo y evitando dependencias circulares.

1.2 Fases y Secuencia de Construcción

El proyecto se estructura en cinco fases de construcción, cada una con objetivos, actividades, entregables y criterios de validación claramente definidos. Las fases no son estrictamente secuenciales en todos los casos: las actividades de testing y documentación son transversales a todas las fases.

Fase	Actividades Principales	Entregables	Dependencias	Validación / Criterio de Éxito
Fase 0 Preparación (Sprint 0)	Configuración de repositorio GitHub. Definición de GitFlow. Configuración de Docker y Docker Compose.	Repositorio configurado. Pipelines CI/CD operativos. Entornos de desarrollo	Ninguna (punto de partida)	Pipeline CI ejecuta build exitoso en rama main. Todos los entornos responden a

	Setup de CI/CD base (GitHub Actions). Configuración de entornos (dev, staging, prod). Definición de estándares de código.	levantados. Documentación de arquitectura técnica.		health-check. Estándares de código documentados y aprobados.
Fase 1 Autenticación & Perfiles	Diseño del modelo de datos de usuarios. Implementación de endpoints de registro, login, logout y recuperación de contraseña. Generación y validación de JWT (RS256). Hash bcrypt. Middleware de autenticación y autorización por roles. Módulo de perfil de usuario.	API de autenticación funcional. Middleware JWT integrado. Tests unitarios y de integración del módulo. Documentación de API (OpenAPI).	Fase 0 completada	Login exitoso retorna JWT válido. Middleware rechaza requests sin token válido (HTTP 401/403). Contraseñas almacenadas como hash bcrypt. Cobertura de tests >= 80%.
Fase 2 Disponibilidad & Agenda	Modelo de datos de agenda, slots y servicios. Implementación CQS: endpoints de consulta separados de comandos. Lógica de cálculo dinámico de slots disponibles. Configuración de horarios y bloqueos manuales. Excepciones de agenda. Caché de disponibilidad.	API de disponibilidad funcional. Módulo de agenda del administrador. Caché de slots implementado. Tests de carga básicos.	Fase 1 completada (JWT requerido para endpoints de escritura)	Consulta de disponibilidad responde en < 2s (RNF-03). CQS correctamente implementado (endpoints Query no modifican estado). Tests de unidad sobre lógica de cálculo de slots al 85%.
Fase 3 Agendamiento & Concurrencia	Modelo de datos de citas y estados. Implementación de reserva, cancelación y reprogramación. Mecanismo anti-doble reserva (bloqueo pesimista SELECT FOR UPDATE). Gestión de estados de cita. Integración con módulo de disponibilidad. Pruebas de concurrencia.	API de agendamiento funcional. Tests de concurrencia (simulación de 50+ usuarios simultáneos). Documentación del mecanismo de bloqueo.	Fase 2 completada (disponibilidad como dependencia directa)	Cero dobles reservas en tests de concurrencia con 100 requests simultáneos sobre el mismo slot (RF-22, RNF-02). Transacciones ACID verificadas. Estados de cita correctamente gestionados.
Fase 4 Notificaciones	Diseño de cola de mensajes asíncrona. Implementación de servicio de correo electrónico (SMTP). Templates de notificación (confirmación, recordatorio, cancelación, reprogramación). Integración mediante eventos de dominio. Sistema de reintentos con backoff exponencial.	Servicio de notificaciones desacoplado. Templates de email implementados. Log de notificaciones. Sistema de reintentos.	Fase 3 completada (eventos de dominio de citas como fuente)	Notificaciones de confirmación enviadas en < 30s tras la reserva. Tasa de entrega >= 98% en ambiente de staging. Fallos encolados y reintentados automáticamente.
Fase 5 Frontend & Integración	Implementación de SPA React/TypeScript. Componentes de calendario, formularios de reserva, panel de administración. Integración con todos los endpoints REST. Gestión de estado y autenticación JWT en cliente. Pruebas E2E. Optimización de performance.	SPA completamente funcional. Flujo de reserva en <= 3 pasos (RNF-20). Pruebas E2E automatizadas. Build de producción optimizado.	Fases 1-4 completadas (APIs como contrato)	Flujo de reserva completado en <= 3 pasos (RNF-20). Diseño responsive verificado en mobile/tablet/desktop (RNF-19). Pruebas E2E al 70% de cobertura de flujos críticos.

1.3 Estrategia de Desarrollo Backend

El backend de la plataforma adopta una arquitectura limpia (Clean Architecture) con inspiración hexagonal, organizada en capas con dependencias unidireccionales hacia el interior. Esta decisión garantiza que la lógica de dominio permanezca completamente aislada de los detalles de infraestructura (base de datos, frameworks, servicios externos), facilitando la mantenibilidad y las pruebas unitarias.

Estructura de Capas del Backend

```
src/
├── domain/                # Capa de Dominio (núcleo, sin dependencias externas)
│   ├── entities/          # Entidades de negocio (Appointment, Schedule, User)
│   ├── value-objects/     # Objetos de valor (TimeSlot, AppointmentStatus)
│   ├── repositories/     # Interfaces de repositorio (contratos, no implementaciones)
│   ├── events/           # Eventos de dominio (AppointmentCreated, AppointmentCancelled)
│   └── exceptions/       # Excepciones de dominio (SlotUnavailableException)
├── application/          # Capa de Aplicación (orquestración de casos de uso)
│   ├── commands/         # COMMAND: handlers que modifican estado
│   │   ├── CreateAppointmentCommand.ts
│   │   ├── CancelAppointmentCommand.ts
│   │   └── ConfigureScheduleCommand.ts
│   ├── queries/          # QUERY: handlers que solo retornan datos
│   │   ├── GetAvailabilitySlotsQuery.ts
│   │   └── GetAppointmentHistoryQuery.ts
│   ├── dtos/             # Data Transfer Objects (Request/Response)
│   ├── services/         # Servicios de aplicación (orquestradores)
│   └── infrastructure/   # Capa de Infraestructura (implementaciones concretas)
│       ├── persistence/  # Implementaciones de repositorios (ORM)
│       ├── http/         # Controladores REST, middlewares
│       ├── messaging/    # Cola de mensajes, event bus
│       ├── email/        # Servicio SMTP, templates
│       ├── cache/        # Redis, estrategias de caché
│       └── shared/       # Código compartido (logger, config, utils)
```

Implementación del Principio CQS en el Backend

El principio Command Query Separation (CQS) se implementa mediante la separación explícita de handlers en dos categorías mutuamente excluyentes. Los Query Handlers únicamente leen el estado del sistema y retornan datos; jamás producen efectos secundarios. Los Command Handlers modifican el estado del sistema y retornan únicamente confirmación de éxito o error, sin datos de negocio. Esta separación se refleja tanto en la estructura de carpetas como en los contratos de las interfaces:

```
// QUERY: Solo lectura, sin efectos secundarios
interface IQueryHandler<TQuery, TResult> {
  handle(query: TQuery): Promise<TResult>;
}

// COMMAND: Modifica estado, retorna void o confirmación
interface ICommandHandler<TCommand> {
  handle(command: TCommand): Promise<void | { id: string }>;
}

// Implementación: CreateAppointmentCommandHandler
class CreateAppointmentCommandHandler implements ICommandHandler<CreateAppointmentCommand> {
  async handle(cmd: CreateAppointmentCommand): Promise<{ id: string }> {
    // 1. Adquirir bloqueo pesimista sobre el slot
    const slot = await this.slotRepo.findByIdForUpdate(cmd.slotId); // SELECT FOR UPDATE
    if (!slot || slot.status !== 'AVAILABLE') {
      throw new SlotUnavailableException(cmd.slotId);
    }
    // 2. Crear entidad de cita y persistir en transacción
    const appointment = Appointment.create(cmd);
    await this.unitOfWork.execute(async (trx) => {
      await this.appointmentRepo.save(appointment, trx);
      await this.slotRepo.markAsReserved(slot.id, trx);
    });
    // 3. Publicar evento de dominio (asíncrono, no bloquea)
    this.eventBus.publish(new AppointmentCreatedEvent(appointment));
    return { id: appointment.id };
  }
}
```

```

}
}

```

Manejo de Concurrencia — Mecanismo Anti-Doble Reserva

El problema de la doble reserva (double booking) bajo alta concurrencia es el desafío técnico más crítico del sistema. Dos usuarios pueden consultar simultáneamente la disponibilidad de un slot, encontrarlo disponible, y proceder a reservarlo al mismo tiempo. Sin un mecanismo de control adecuado, ambas reservas serían confirmadas sobre el mismo bloque horario, violando RF-22 y RNF-02.

La solución implementada combina dos mecanismos complementarios:

- **Bloqueo Pesimista (Pessimistic Locking):** La operación de reserva inicia con un `SELECT FOR UPDATE` sobre el registro del slot horario. Esta instrucción adquiere un bloqueo exclusivo a nivel de fila en la base de datos, garantizando que ninguna otra transacción concurrente pueda leer ni modificar ese slot hasta que la transacción actual sea confirmada (`COMMIT`) o revertida (`ROLLBACK`). Este mecanismo es apropiado para sistemas donde la probabilidad de conflicto es alta (múltiples usuarios reservando los mismos slots populares).
- **Constraint de unicidad en base de datos:** Como segunda línea de defensa, se implementa un constraint `UNIQUE` sobre la combinación (`schedule_slot_id`, `status`) filtrado únicamente para `status IN ('CONFIRMED', 'PENDING')`, garantizando integridad a nivel de base de datos independientemente de la lógica de aplicación.

```

-- Constraint de unicidad como segunda línea de defensa
CREATE UNIQUE INDEX idx_unique_active_appointment
ON appointments (schedule_slot_id)
WHERE status IN ('CONFIRMED', 'PENDING');

-- Operación de reserva atómica
BEGIN;
  SELECT * FROM schedule_slots
  WHERE id = $1 AND status = 'AVAILABLE'
  FOR UPDATE NOWAIT; -- Falla inmediatamente si el slot está bloqueado
  INSERT INTO appointments (...) VALUES (...);
  UPDATE schedule_slots SET status = 'RESERVED' WHERE id = $1;
COMMIT;

```

1.4 Estrategia de Desarrollo Frontend

El frontend se implementa como una Single Page Application (SPA) construida con React 18 y TypeScript, aprovechando el sistema de tipos estático para garantizar contratos claros entre componentes y servicios de API. La arquitectura frontend sigue el patrón de separación de responsabilidades: componentes de presentación (Presentational Components) desacoplados de la lógica de negocio, la cual reside en hooks personalizados y servicios de aplicación cliente.

Estructura del Proyecto Frontend

```

src/
├── features/
│   ├── auth/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── services/
│   ├── booking/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── services/
│   ├── schedule/
│   └── profile/
├── shared/
│   ├── components/
│   ├── hooks/
│   ├── services/
│   └── types/
└── router/

```

Funcionalidades organizadas por dominio
Login, register, password recovery
AuthForm, LoginPage, RegisterPage
useAuth, useJWT
authService (llamadas API)
Flujo de reserva (máx. 3 pasos - RNF-20)
ServiceSelector, CalendarPicker, ConfirmStep
useBooking, useAvailability
bookingService
Panel de agenda (Administrador)
Gestión de perfil
Button, Modal, Spinner (reutilizables)
useApi, useToast
httpClient (axios instance con interceptors)
Tipos TypeScript compartidos
React Router: rutas públicas y protegidas

```
└─ store/ # Estado global (Context API o Zustand)
```

Autenticación JWT en el Cliente

La gestión del token JWT en el frontend implementa las siguientes prácticas de seguridad: el Access Token (expiración 15 min) se almacena en memoria del runtime de JavaScript (no en localStorage ni cookies sin HttpOnly) para mitigar ataques XSS. El Refresh Token se almacena en una cookie HttpOnly, inaccesible desde JavaScript. Un interceptor de Axios intercepta automáticamente los errores HTTP 401 y ejecuta el flujo de renovación de token transparente para el usuario:

```
// Interceptor de Axios para renovación automática de token
httpClient.interceptors.response.use(
  response => response,
  async error => {
    const originalRequest = error.config;
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      try {
        const { accessToken } = await authService.refreshToken();
        tokenStore.setAccessToken(accessToken);
        originalRequest.headers['Authorization'] = `Bearer ${accessToken}`;
        return httpClient(originalRequest);
      } catch {
        tokenStore.clear();
        router.navigate('/login');
      }
    }
    return Promise.reject(error);
  }
);
```

Flujo de Reserva en 3 Pasos (RNF-20)

El proceso de agendamiento cumple estrictamente el requerimiento RNF-20: no más de 3 pasos principales en la interfaz de usuario. La implementación utiliza un componente Stepper con estado centralizado:

- Paso 1 — Selección: El usuario selecciona el servicio y visualiza el calendario de disponibilidad en tiempo real.
- Paso 2 — Confirmación: El sistema muestra el resumen de la reserva (servicio, profesional, fecha, hora, duración estimada). El usuario verifica los datos.
- Paso 3 — Resultado: El sistema procesa la reserva y muestra confirmación exitosa con el código de reserva o el mensaje de error en caso de conflicto de concurrencia, ofreciendo la opción de seleccionar otro horario.

1.5 Estrategia de Base de Datos

La base de datos del sistema utiliza PostgreSQL 15 como motor relacional principal, seleccionado por su soporte nativo de transacciones ACID, bloqueos a nivel de fila (SELECT FOR UPDATE), índices parciales, tipos de datos avanzados (JSONB para metadatos) y su robustez probada en aplicaciones de alta concurrencia. El modelo de datos es diseñado con integridad referencial explícita, constraints de unicidad y políticas de indexación optimizadas para los patrones de consulta del sistema.

Modelo Relacional Principal

```
-- Tabla de usuarios (base del sistema de autenticación)
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name   VARCHAR(255) NOT NULL,
  role        VARCHAR(20)  NOT NULL DEFAULT 'CLIENT' CHECK (role IN ('CLIENT', 'ADMIN')),
  status      VARCHAR(20)  NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING', 'ACTIVE', 'BLOCKED'));
```

```

    created_at    TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

-- Tabla de servicios ofrecidos
CREATE TABLE services (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(255) NOT NULL,
    description    TEXT,
    duration_min  INTEGER NOT NULL CHECK (duration_min > 0),
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,
    created_by    UUID REFERENCES users(id),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabla de configuración de agenda (horarios del negocio)
CREATE TABLE schedule_configs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    service_id    UUID NOT NULL REFERENCES services(id),
    admin_id      UUID NOT NULL REFERENCES users(id),
    day_of_week   SMALLINT NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
    open_time     TIME NOT NULL,
    close_time    TIME NOT NULL,
    slot_duration INTEGER NOT NULL,
    buffer_min    INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT chk_times CHECK (open_time < close_time)
);

-- Tabla de slots horarios (generados a partir de schedule_config)
CREATE TABLE schedule_slots (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    config_id     UUID NOT NULL REFERENCES schedule_configs(id),
    slot_date     DATE NOT NULL,
    start_time    TIME NOT NULL,
    end_time      TIME NOT NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE'
                CHECK (status IN ('AVAILABLE', 'RESERVED', 'BLOCKED')),
    version       INTEGER NOT NULL DEFAULT 0, -- Para Optimistic Locking
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabla de citas (entidad central del negocio)
CREATE TABLE appointments (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_id     UUID NOT NULL REFERENCES users(id),
    slot_id       UUID NOT NULL REFERENCES schedule_slots(id),
    service_id    UUID NOT NULL REFERENCES services(id),
    status        VARCHAR(20) NOT NULL DEFAULT 'CONFIRMED'
                CHECK (status IN ('PENDING', 'CONFIRMED', 'CANCELLED', 'RESCHEDULED', 'COMPLETED')),
    notes        TEXT,
    cancelled_by  UUID REFERENCES users(id),
    cancelled_at  TIMESTAMPTZ,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Constraint crítico anti-doble reserva
CREATE UNIQUE INDEX idx_unique_active_slot
ON appointments (slot_id)
WHERE status IN ('CONFIRMED', 'PENDING');

-- Índices optimizados para patrones de consulta frecuentes
CREATE INDEX idx_slots_date_status ON schedule_slots (slot_date, status);
CREATE INDEX idx_appointments_client ON appointments (client_id, status, created_at DESC);
CREATE INDEX idx_appointments_slot ON appointments (slot_id);
CREATE INDEX idx_users_email ON users (email);

```

Estrategia de Migraciones

Las migraciones de base de datos se gestionan mediante una herramienta de migración versionada (Flyway o herramienta equivalente del ORM seleccionado). Cada migración es un archivo SQL inmutable nombrado con

convención V{versión}__{descripción}.sql. Las migraciones se ejecutan automáticamente durante el proceso de despliegue, antes del inicio de la aplicación. Las migraciones de rollback (U{versión}__{descripción}.sql) están definidas para las migraciones críticas, permitiendo revertir cambios de esquema en caso de fallos de despliegue.

1.6 Gestión de Entornos

El sistema opera en cuatro entornos diferenciados, cada uno con configuración, datos y propósito específicos. La segmentación de entornos garantiza que los cambios sean validados progresivamente antes de llegar a producción, minimizando el riesgo de incidentes.

Entorno	Propósito	Base de Datos	Deploy Trigger	Configuración Clave
LOCAL	Desarrollo individual	PostgreSQL en Docker (datos ficticios)	Manual (docker compose up)	DEBUG=true, JWT_SECRET=dev-secret, LOG_LEVEL=debug
DEV	Integración continua del equipo	PostgreSQL compartido (datos de prueba)	Push a rama develop	CI/CD auto-deploy, seeding de datos de prueba
STAGING	Validación pre-producción y UAT	Réplica de producción (datos anonimizados)	Merge a rama release	Configuración idéntica a producción, SSL activo
PRODUCCIÓN	Operación real del sistema	PostgreSQL con backups automáticos y réplica	Tag de versión en main	JWT_SECRET rotado, HTTPS obligatorio, logs auditables

Variables de Entorno y Gestión de Secretos

```
# .env.production (valores reales gestionados por GitHub Secrets / Vault)

# Base de datos
DATABASE_URL=postgres://user:PASSWORD@db-host:5432/reservas_db
DATABASE_POOL_SIZE=20
DATABASE_SSL=true

# Autenticación JWT
JWT_PRIVATE_KEY_PATH=/run/secrets/jwt_private_key
JWT_PUBLIC_KEY_PATH=/run/secrets/jwt_public_key
JWT_ACCESS_EXPIRY=900           # 15 minutos en segundos
JWT_REFRESH_EXPIRY=604800       # 7 días en segundos

# Redis (caché y blacklist JWT)
REDIS_URL=redis://redis-host:6379
REDIS_PASSWORD=REDIS_SECRET

# Servicio de correo
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_API_KEY=SG.XXXXXX
MAIL_FROM=noreply@reservas-platform.com

# Aplicación
NODE_ENV=production
PORT=3000
LOG_LEVEL=info
CORS_ORIGINS=https://reservas-platform.com
```

1.7 Gestión de Código Fuente — GitFlow

El proyecto adopta el modelo de ramificación GitFlow, estándar de la industria para proyectos con ciclos de entrega regulares. Este modelo provee estructura y predictibilidad al proceso de integración de código, reduciendo conflictos de merge y garantizando la estabilidad de las ramas principales.

Estructura de Ramas

```
Ramas permanentes:
  main          → Código en producción. Protegida. Solo acepta merges desde release/* y hotfix/*
  develop       → Integración continua. Acepta merges desde feature/* y bugfix/*

Ramas temporales:
  feature/{ticket}-{descripcion} → Nuevas funcionalidades (ej: feature/US-15-reservar-cita)
  bugfix/{ticket}-{descripcion}  → Corrección de bugs no críticos
  release/{version}              → Preparación de release (ej: release/1.2.0)
  hotfix/{ticket}-{descripcion}  → Corrección urgente en producción
```

Convenciones de Commits (Conventional Commits)

```
formato: <tipo>(<alcance>): <descripción breve>

Tipos aceptados:
  feat:      Nueva funcionalidad
  fix:       Corrección de bug
  refactor:  Refactorización sin cambio de comportamiento
  test:      Adición o modificación de tests
  docs:      Documentación
  chore:     Tareas de mantenimiento (deps, config)

Ejemplos válidos:
  feat(booking): implement pessimistic locking for slot reservation
  fix(auth): correct JWT expiration validation in middleware
  test(availability): add concurrency tests for double-booking prevention
```

Política de Pull Requests y Code Review

- Toda contribución de código debe realizarse mediante Pull Request (PR) hacia la rama destino correspondiente.
- Todo PR requiere revisión y aprobación de al menos un integrante del equipo distinto al autor antes de ser mergeado.
- Los PR deben incluir: descripción del cambio, issue/ticket relacionado, checklist de calidad y capturas de pantalla o evidencia de pruebas para cambios de UI.
- La rama main está protegida: no se permiten pushes directos. Solo merges desde release o hotfix aprobados por el líder técnico.
- Los checks de CI (build, tests, linting) deben pasar exitosamente antes de habilitar el merge.

1.8 Roles y Responsabilidades — Matriz RACI

La siguiente matriz RACI (Responsible, Accountable, Consulted, Informed) define claramente las responsabilidades de cada rol en el proyecto para las actividades principales del ciclo de desarrollo y despliegue. R = Responsable de ejecutar, A = Aprobador final (Accountable), C = Consultado, I = Informado.

Actividad / Entregable	Arquitecto	Backend Dev	Frontend Dev	QA	DevOps	DBA	Líder Téc.
Diseño de Arquitectura	A	C	C	I	C	C	R
Implementación Backend	C	R	I	I	I	C	A

Implementación Frontend	C	I	R	I	I	I	A
Diseño del Modelo de Datos	C	C	I	I	I	R	A
Configuración CI/CD	C	C	C	I	R	I	A
Escritura de Tests Unitarios	I	R	R	I	I	I	A
Ejecución de Tests de Integración	I	C	C	R	C	C	A
Pruebas de Concurrencia	C	C	I	R	C	I	A
Despliegue a Staging	C	C	C	I	R	I	A
Despliegue a Producción	A	C	C	I	R	C	R
Monitoreo y Alertas	I	I	I	I	R	I	A
Revisión de Code Reviews	C	R	R	I	C	I	A
Gestión de Backups BD	I	I	I	I	C	R	A
Respuesta a Incidentes P1	A	C	C	I	R	C	R

1.9 Estrategia de Testing

La estrategia de testing adopta el modelo de pirámide de pruebas: mayor cantidad de tests unitarios (base de la pirámide), menor cantidad de tests de integración (nivel medio) y un conjunto focalizado de tests end-to-end y de sistema (cúspide). Esta distribución optimiza el tiempo de ejecución del feedback loop y reduce el costo de mantenimiento del suite de pruebas.

Nivel	Tipo de Test	Herramientas	Cobertura Objetivo	Frecuencia
Unitario	Lógica de dominio, handlers CQS, validaciones	Jest / Vitest, Testing Library	>= 85% de líneas en capa domain y application	Cada commit (CI)
Integración	Repositorios con BD real, endpoints REST	Supertest, TestContainers (PostgreSQL en Docker)	>= 75% de flujos de API	Cada PR (CI)
Concurrencia	Prevención de doble reserva, race conditions	k6, Artillery, scripts personalizados	100 req. simultáneos → 0 dobles reservas	Previo a cada release
Seguridad	SQL Injection, XSS, autenticación JWT	OWASP ZAP, análisis estático (ESLint security)	0 vulnerabilidades críticas o altas	Cada release
E2E	Flujos completos usuario-sistema	Playwright / Cypress	>= 70% flujos críticos (reserva, login, cancelación)	Nightly build
Carga / Estrés	Rendimiento bajo carga sostenida y pico	k6, Grafana (monitoreo durante prueba)	P95 < 2s con 500 usuarios concurrentes	Previo a producción

1.10 Gestión de Riesgos Técnicos

La gestión de riesgos es un proceso continuo que acompaña todo el ciclo de vida del proyecto. Los riesgos identificados a continuación han sido evaluados según probabilidad de ocurrencia e impacto en los objetivos del

proyecto, clasificándose en tres niveles: Alto (requiere plan de mitigación activo), Medio (requiere monitoreo continuo) y Bajo (aceptable con medidas preventivas básicas).

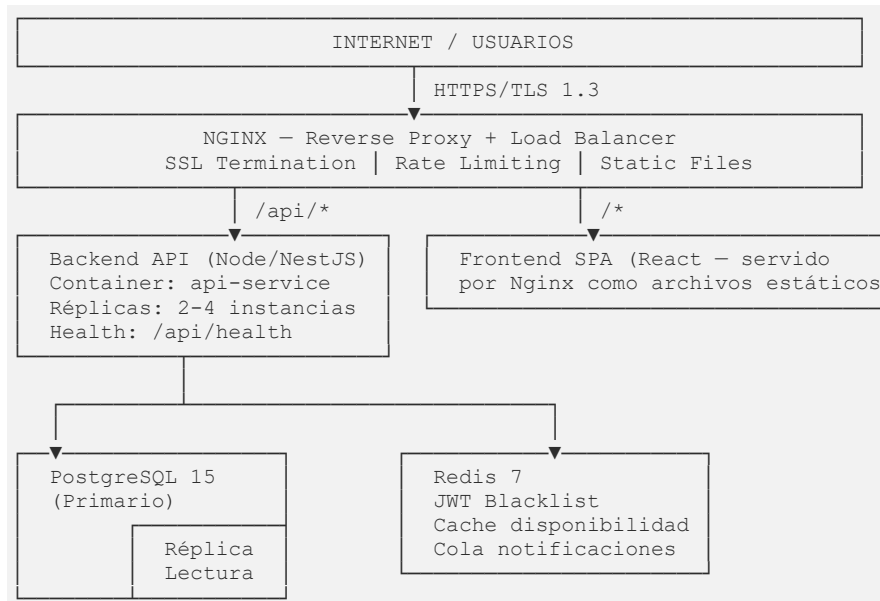
Riesgo	Prob.	Impacto	Nivel	Estrategia de Mitigación	Plan de Contingencia
Doble reserva bajo alta concurrencia	Alta	Alto	Alto	Bloqueo pesimista (SELECT FOR UPDATE) + constraint UNIQUE en BD. Tests de concurrencia obligatorios previo a cada release.	Rollback de transacción automático. Notificación al cliente con oferta de horario alternativo.
Expiración masiva de tokens JWT en producción	Media	Medio	Medio	Access Token con TTL de 15 min + Refresh Token. Renovación silenciosa implementada en cliente. Monitoreo de errores 401.	Forzar re-login masivo con mensaje informativo. Ajuste de TTL según métricas de sesión.
Caída del servicio de base de datos	Baja	Alto	Alto	Réplica de lectura activa. Healthchecks con reintentos automáticos. Circuit Breaker pattern en capa de repositorio.	Failover automático a réplica. Activación de modo degradado (solo lectura). Restauración desde backup en < 30 min.
Fallo del servicio de correo electrónico	Alta	Medio	Medio	Cola de mensajes asíncrona (retry con backoff exponencial). Múltiples proveedores SMTP configurados (primario + fallback).	Mensajes encolados para reintento. Alerta al administrador. Reenvío manual desde panel de admin.
Deuda técnica acumulada por presión de plazos	Media	Medio	Medio	Code reviews obligatorios. Definition of Done incluye cobertura de tests mínima. Sprints con capacidad de refactorización.	Sprint dedicado a refactorización. Revisión de arquitectura con el arquitecto del equipo.
Incompatibilidad de dependencias o breaking changes	Media	Medio	Medio	Lockfiles (package-lock.json) en versionamiento. Actualizaciones de dependencias en rama separada con CI completo.	Rollback de versión de dependencia. Mantenimiento de versión LTS hasta resolución.
Brecha de seguridad en endpoints REST	Baja	Alto	Alto	OWASP Top 10 como checklist de revisión. Análisis estático de seguridad en CI. Validación de inputs en capa de aplicación.	Despliegue de hotfix en < 2 horas. Rotación de secrets comprometidos. Notificación a usuarios afectados.
Degradación de rendimiento con crecimiento de datos	Media	Medio	Medio	Índices optimizados desde el diseño inicial. Monitoreo de slow queries. Caché de consultas frecuentes (Redis).	Análisis de query execution plans. Particionamiento de tablas de alto volumen. Escala horizontal.

PARTE 2: GUÍA DE DESPLIEGUE Y OPERACIONES

2.1 Arquitectura de Infraestructura

La infraestructura de producción del sistema es diseñada para cumplir simultáneamente tres objetivos fundamentales: disponibilidad operativa $\geq 99.5\%$ anual (RNF-13), tiempo de respuesta ≤ 2 segundos en consultas de disponibilidad (RNF-03) y capacidad de escalabilidad horizontal (RNF-01). La arquitectura adopta el modelo de contenedores orquestados, con separación clara entre los niveles de presentación, procesamiento y persistencia.

Componentes de Infraestructura



Justificación de Disponibilidad 99.5%

El objetivo de disponibilidad RNF-13 ($\geq 99.5\%$ anual, equivalente a un máximo de 43.8 horas de downtime anuales) se alcanza mediante: (a) redundancia de instancias del backend (mínimo 2 réplicas activas), (b) reverse proxy Nginx con healthcheck automático que redirige el tráfico ante fallo de una instancia, (c) réplica de lectura en PostgreSQL para garantizar disponibilidad en caso de fallo del nodo primario, y (d) mecanismo automático de recuperación ante fallos (RNF-14) mediante políticas de restart en Docker.

Certificado TLS y Seguridad de Red

- Certificados TLS gestionados mediante Let's Encrypt con renovación automática (certbot --standalone).
- Redirección obligatoria HTTP $\rightarrow$ HTTPS en Nginx (return 301).
- Cabeceras de seguridad configuradas: HSTS, X-Frame-Options, X-Content-Type-Options, CSP.
- Firewall: únicamente puertos 80 (redirect) y 443 (HTTPS) expuestos públicamente. Puerto 22 (SSH) restringido a IPs de administración.

2.2 Estrategia CI/CD — GitHub Actions

El pipeline de integración y despliegue continuo está implementado en GitHub Actions, seleccionado por su integración nativa con el repositorio GitHub del proyecto, su modelo de configuración como código (YAML en `.github/workflows/`) y su capacidad de ejecutar jobs en paralelo para optimizar los tiempos del pipeline. El pipeline diferencia entre el flujo de CI (ejecutado en cada PR y push a develop) y el flujo de CD (ejecutado únicamente en tags de versión sobre main).

Pipeline CI — Integración Continua

```
# .github/workflows/ci.yml
name: Continuous Integration

on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop]

jobs:
  quality-gate:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: reservas_test
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        ports: ['5432:5432']
        options: --health-cmd pg_isready --health-interval 10s
      redis:
        image: redis:7-alpine
        ports: ['6379:6379']

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint (ESLint + Prettier)
        run: npm run lint

      - name: Type checking (TypeScript)
        run: npm run type-check

      - name: Run unit tests
        run: npm run test:unit -- --coverage

      - name: Run integration tests
        run: npm run test:integration
        env:
          DATABASE_URL: postgresql://test:test@localhost:5432/reservas_test
          REDIS_URL: redis://localhost:6379

      - name: Quality Gate - Coverage
        run: |
          COVERAGE=$(cat coverage/coverage-summary.json | jq '.total.lines.pct')
          if (( $(echo "$COVERAGE < 80" | bc -l) )); then
            echo 'FAIL: Coverage below 80%: '$COVERAGE
            exit 1
          fi

      - name: Security audit
```

```
run: npm audit --audit-level=high

- name: Build Docker image (validation only)
  run: docker build -t reservas-api:ci-test .
```

Pipeline CD — Despliegue Continuo

```
# .github/workflows/cd.yml
name: Continuous Deployment

on:
  push:
    tags: ['v*.*.*'] # Solo en tags de versión semántica

jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment: production # Requiere aprobación manual
    needs: [quality-gate] # CI debe pasar primero

    steps:
      - uses: actions/checkout@v4

      - name: Log in to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: |
            ghcr.io/${ github.repository }/api:${ github.ref_name }
            ghcr.io/${ github.repository }/api:latest

      - name: Deploy to production server
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${ secrets.PROD_HOST }
          username: ${ secrets.PROD_USER }
          key: ${ secrets.PROD_SSH_KEY }
          script: |
            cd /opt/reservas-platform
            docker compose pull
            docker compose up -d --no-deps --scale api=2 api
            docker compose exec api npm run db:migrate
            docker compose ps
```

2.3 Procedimiento Paso a Paso de Despliegue

Docker Compose — Configuración de Producción

```
# docker-compose.prod.yml
version: '3.9'

services:
  nginx:
    image: nginx:1.25-alpine
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certbot/conf:/etc/letsencrypt:ro
      - frontend-build:/usr/share/nginx/html:ro
```

```

depends_on: [api]
restart: unless-stopped

api:
  image: ghcr.io/org/reservas-api:${APP_VERSION:-latest}
  deploy:
    replicas: 2
    update_config:
      parallelism: 1
      delay: 30s
      failure_action: rollback
    restart_policy:
      condition: on-failure
      max_attempts: 3
  environment:
    - NODE_ENV=production
    - DATABASE_URL=${DATABASE_URL}
    - REDIS_URL=${REDIS_URL}
    - JWT_PRIVATE_KEY_PATH=/run/secrets/jwt_private
  secrets:
    - jwt_private
    - jwt_public
  healthcheck:
    test: ['CMD', 'curl', '-f', 'http://localhost:3000/api/health']
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_healthy }

postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - postgres-data:/var/lib/postgresql/data
    - ./backups:/backups
  healthcheck:
    test: ['CMD-SHELL', 'pg_isready -U ${POSTGRES_USER}']
    interval: 10s
    timeout: 5s
    retries: 5
  restart: unless-stopped

redis:
  image: redis:7-alpine
  command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
  volumes:
    - redis-data:/data
  healthcheck:
    test: ['CMD', 'redis-cli', '--pass', '${REDIS_PASSWORD}', 'ping']
    interval: 10s
  restart: unless-stopped

notification-worker:
  image: ghcr.io/org/reservas-api:${APP_VERSION:-latest}
  command: node dist/workers/notification.worker.js
  environment:
    - NODE_ENV=production
    - REDIS_URL=${REDIS_URL}
    - SMTP_API_KEY=${SMTP_API_KEY}
  depends_on: [redis]
  restart: unless-stopped

secrets:
  jwt_private:
    file: ./secrets/jwt_private.pem
  jwt_public:
    file: ./secrets/jwt_public.pem

```

```
volumes:
  postgres-data:
  redis-data:
  frontend-build:
```

Dockerfile del Backend

```
# Dockerfile - Multi-stage build para minimizar imagen de producción
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY . .
RUN npm run build

# Imagen de producción (sin devDependencies ni código fuente TypeScript)
FROM node:20-alpine AS production
RUN addgroup -g 1001 -S nodejs && adduser -S nodejs -u 1001
WORKDIR /app
COPY --from=builder --chown=nodejs:nodejs /app/dist ./dist
COPY --from=builder --chown=nodejs:nodejs /app/node_modules ./node_modules
COPY --from=builder --chown=nodejs:nodejs /app/package.json ./
USER nodejs
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD curl -f http://localhost:3000/api/health || exit 1
CMD ["node", "dist/main.js"]
```

Secuencia de Despliegue Manual en Producción

1. Verificación previa: Confirmar que el pipeline CI ha pasado exitosamente para el tag a desplegar.
2. Backup previo al despliegue: Ejecutar backup manual de la base de datos antes de cualquier cambio.
3. Pull de nuevas imágenes: `docker compose -f docker-compose.prod.yml pull`
4. Ejecución de migraciones de base de datos: `docker compose exec api npm run db:migrate`
5. Despliegue con zero-downtime (rolling update): `docker compose up -d --no-deps --scale api=2 api`
6. Verificación de healthchecks: `docker compose ps` — esperar estado 'healthy' en todos los contenedores.
7. Smoke tests post-despliegue: Ejecutar suite de tests de humo contra el endpoint de producción.
8. Monitoreo activo por 15 minutos: Observar dashboards de Grafana para detectar anomalías en métricas.

2.4 Estrategia de Rollback

La estrategia de rollback define los procedimientos para revertir el sistema a un estado estable previo ante fallos detectados durante o después del despliegue. Los criterios de activación del rollback son: tasa de error HTTP 5xx > 5% sostenida por más de 2 minutos, tiempo de respuesta P95 > 4 segundos, fallo de healthcheck en más del 50% de instancias, o anomalías críticas detectadas en el monitoreo post-despliegue.

Rollback del Backend (< 5 minutos)

```
# 1. Identificar la versión estable anterior
docker images ghcr.io/org/reservas-api --format '{{.Tag}}\t{{.CreatedAt}}'

# 2. Actualizar variable de versión y redespargar imagen anterior
export APP_VERSION=v1.2.3 # Versión estable anterior
docker compose -f docker-compose.prod.yml up -d --no-deps api

# 3. Verificar estado post-rollback
docker compose ps
curl -f https://api.reservas-platform.com/api/health
```


Rollback de Base de Datos

El rollback de base de datos es la operación más delicada y requiere mayor cuidado dado que implica posible pérdida de datos transaccionales ocurridos entre el despliegue y el rollback. El proceso diferencia entre rollback de migraciones no destructivas (reversibles) y rollback de migraciones destructivas (requieren restauración desde backup):

```
# Rollback de migración no destructiva (añadir columna, añadir índice)
docker compose exec api npm run db:migrate:undo

# Rollback destructivo – Restauración desde backup (último backup previo al deploy)
# ADVERTENCIA: Esta operación implica pérdida de transacciones post-backup

# 1. Detener la aplicación para evitar escrituras durante la restauración
docker compose stop api notification-worker

# 2. Crear backup del estado actual (por seguridad)
docker compose exec postgres pg_dump -U $POSTGRES_USER $POSTGRES_DB \
  > /backups/emergency-$(date +%Y%m%d-%H%M%S).sql

# 3. Restaurar desde backup previo al despliegue
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB \
  -c 'DROP SCHEMA public CASCADE; CREATE SCHEMA public;'
docker compose exec postgres psql -U $POSTGRES_USER $POSTGRES_DB \
  < /backups/pre-deploy-backup.sql

# 4. Reiniciar aplicación con imagen anterior
export APP_VERSION=v1.2.3
docker compose -f docker-compose.prod.yml up -d
```

2.5 Operaciones y Monitoreo

La estrategia de observabilidad del sistema se articula sobre tres pilares fundamentales: Logs (registro estructurado de eventos), Métricas (indicadores cuantitativos de salud del sistema) y Trazas (seguimiento de requests a través de los componentes del sistema). Esta trinidad de observabilidad permite identificar, diagnosticar y resolver incidentes de forma rápida y fundamentada.

Stack de Monitoreo

- Prometheus: Recolección de métricas de la API (latencia, tasa de errores, throughput, métricas de negocio como reservas por hora) mediante endpoint /metrics en formato Prometheus.
- Grafana: Visualización de dashboards operativos. Dashboards pre-configurados para: rendimiento de API, estado de base de datos, métricas de concurrencia de reservas, tasas de éxito de notificaciones y auditoría de autenticación JWT.
- ELK Stack (Elasticsearch, Logstash, Kibana): Centralización y análisis de logs estructurados en formato JSON. Permite búsquedas complejas sobre logs de auditoría, errores y eventos de negocio.
- Alertmanager: Gestión de alertas y notificaciones al equipo de operaciones mediante canales configurables (email, Slack).

Métricas Críticas Monitoreadas

Métrica	Descripción	Umbral de Alerta	Acción Automática
http_request_duration_p95	Percentil 95 de latencia de respuesta	> 2000ms (RNF-03)	Alerta PagerDuty + escalamiento
http_error_rate_5xx	Tasa de errores HTTP 5xx sobre total de requests	> 5%	Alerta crítica + inicio de runbook de incidente

appointment_double_booking_attempts	Intentos de doble reserva detectados y bloqueados	> 10 en 5 min	Alerta de investigación de posible ataque
db_connection_pool_usage	Porcentaje de conexiones del pool en uso	> 80%	Alerta + análisis de slow queries activos
jwt_blacklist_size	Número de tokens JWT invalidados en Redis	Crecimiento anómalo	Alerta + verificación de posible ataque de logout masivo
notification_delivery_rate	Tasa de entrega exitosa de notificaciones por email	< 95%	Alerta + verificación del servicio SMTP externo

Configuración de Logging Estructurado

```
// Logger configurado con Winston – Formato JSON para ELK
const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json() // Formato estructurado para Logstash
  ),
  defaultMeta: {
    service: 'reservas-api',
    version: process.env.APP_VERSION
  },
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: '/logs/app.log', maxsize: 10485760 })
  ]
});

// Ejemplo de log de auditoría de reserva
logger.info('appointment.created', {
  appointmentId: appointment.id,
  clientId:      command.clientId,
  slotId:       command.slotId,
  serviceId:    command.serviceId,
  duration_ms:  Date.now() - startTime,
  correlationId: request.headers['x-correlation-id']
});
```

2.6 Respaldo y Recuperación

La estrategia de respaldo implementa el modelo 3-2-1: tres copias de los datos, en dos tipos de medios distintos, con una copia offsite. Esta estrategia garantiza la recuperabilidad de la información ante los escenarios de fallo más severos, incluyendo corrupción de datos, eliminación accidental y fallo catastrófico del servidor.

Tipo de Backup	Herramienta	Frecuencia	Retención	Almacenamiento	RPO Objetivo
Full BD	pg_dump	Diario (02:00 AM)	30 días	S3 / local	24 horas
Incremental BD	WAL archiving	Continuo (streaming)	7 días	S3	< 1 hora
Pre-deploy	pg_dump manual	Antes de cada deploy	14 días	Local + S3	< 30 min
Snapshots VM	Cloud provider	Semanal	4 semanas	Cloud	1 semana

Script de Backup Automático

```
#!/bin/bash
# /opt/scripts/backup-db.sh - Ejecutado por cron a las 02:00 AM

TIMESTAMP=$(date +%Y%m%d-%H%M%S)
BACKUP_FILE="/backups/reservas-$TIMESTAMP.sql.gz"
S3_BUCKET="s3://reservas-platform-backups"

echo "[$(date)] Iniciando backup de base de datos..."

# Dump comprimido
docker compose exec -T postgres pg_dump \
-U $POSTGRES_USER $POSTGRES_DB | gzip > $BACKUP_FILE

if [ $? -eq 0 ]; then
    echo "[$(date)] Backup local exitoso: $BACKUP_FILE"

    # Subir a S3 (si está configurado)
    aws s3 cp $BACKUP_FILE $S3_BUCKET/ --storage-class STANDARD_IA

    # Eliminar backups locales con más de 30 días
    find /backups -name '*.sql.gz' -mtime +30 -delete

    echo "[$(date)] Backup completado exitosamente."
else
    echo "[$(date)] ERROR: Fallo en backup. Enviando alerta..."
    # Trigger alerta - notificación al equipo de ops
fi
```

2.7 Gestión de Incidentes

La gestión de incidentes define los procedimientos para detectar, clasificar, responder y resolver interrupciones del servicio de forma ordenada y eficiente. El objetivo es minimizar el tiempo de restauración del servicio (MTTR - Mean Time To Recovery) y el impacto sobre los usuarios.

Clasificación de Incidentes y SLA de Respuesta

P	Prioridad	Criterio de Clasificación	Tiempo de Respuesta	Tiempo de Resolución	Escalamiento
P1	CRÍTICO	Sistema completamente no disponible. Pérdida de datos activa. Brecha de seguridad confirmada.	< 15 minutos	< 2 horas	Líder Técnico + DevOps inmediato
P2	ALTO	Módulo crítico degradado (reservas, autenticación). Errores 5xx > 5%.	< 30 minutos	< 4 horas	DevOps + Backend Dev
P3	MEDIO	Funcionalidad no crítica afectada (notificaciones, perfil). Degradación de performance.	< 2 horas	< 24 horas	Backend/Frontend Dev
P4	BAJO	Bugs menores, mejoras de UX, incidencias estéticas sin impacto funcional.	< 8 horas	Próximo sprint	Dev asignado

Runbook — Incidente P1: Sistema No Disponible

9. Detección (T+0): Alerta automática recibida por Prometheus/Alertmanager. Confirmación manual accediendo a <https://reservas-platform.com>.
10. Notificación inmediata (T+5 min): Activar canal de incidente en Slack (#incidents). Notificar al Líder Técnico y DevOps. Crear ticket de incidente con timestamp.
11. Diagnóstico inicial (T+10 min): `docker compose ps` (estado de contenedores), `docker compose logs --tail=100 api` (últimos errores), verificación de métricas en Grafana.
12. Acción de mitigación (T+15 min): Si es fallo de código → iniciar rollback de imagen anterior. Si es fallo de BD → failover a réplica. Si es fallo de red → verificar Nginx y certificados TLS.
13. Monitoreo de restauración (T+30 min): Verificar healthchecks, ejecutar smoke tests, confirmar retorno a parámetros normales en Grafana.
14. Comunicación a usuarios (T+30 min): Publicar status en página de estado del servicio.
15. Post-mortem (T+48 horas): Documentar causa raíz, línea de tiempo, acciones tomadas y medidas preventivas. Compartir con el equipo completo.